Actual hardware controllers currently include released "002" revision firmware on revision "D" hardware. Proceed with research & design of Pi-host WifiStepper emulation over UART.

Your hardware detail changes the research baseline: the Pi host should target **stable-002** firmware on revision D boards, not the later `micronexx` version 6 source previously inspected.

I located the release-era source commit explicitly labeled “Updated documentation and firmware to version 2,” verified its `BRANCH "stable"` / `VERSION (2)` declarations, and re-read its UART implementation and binary structure definitions. The core Pi-master approach remains viable, but it should now be designed against that pinned source with an initial compatibility check against your actual controllers. [1]

1. Establishing the correct firmware baseline

The repository’s release manifest identifies:

Item	Release metadata
Model	`wsx100`
Branch	`stable`
Version	`2`
Firmware download	`wifistepper.image.2.bin`
Published SHA-1	`2b76f44c368a5515ca71fdc2de29c8de3522f2f8`
Release-associated source commit	`3627d32739669bb249ec7a8760bd522925041a52`

The manifest’s version-2 changelog includes command-name consistency changes, addition of `setsignal` and `incsignal`, and exposure of all commands through REST. The project distributes these images through its own download page rather than GitHub Releases; the GitHub release and tag lists are empty. [2] [1]

There is one important evidence boundary: this is a strong source-to-release association, not proof of a reproducible, byte-identical build of the released binary. The host should verify the controller’s reported identity and packet lengths before enabling anything beyond discovery and observation.

Difference from the previous investigation

The version-2 UART source confirms the previously identified:

- 512000-baud serial link.
- `0xAB` frame magic and additive checksums.
- 12-byte header with a 32-bit command ID.
- Ring-based discovery and return addressing.
- Periodic configuration and state reports.
- Separate command acknowledgment and execution paths.[1]

However, later additions must not leak into the version-2 protocol profile. For example, the later UART `STOPSWITCH` command is absent from the inspected version-2 `daisy.cpp`.[1]

2. Proposed scope: a small host, not a replacement controller stack

The first implementation should be a standalone Python 3 serial probe with three operating levels:

Level	Permitted behavior	Purpose
Passive capture	Receive and record only	Inspect startup text, electrical behavior, and unsolicited traffic
Discovery	Send only canonical discovery pings	Establish a complete UART loop and slave count
Observe	Discovery plus `SYNC` requests	Identify firmware and decode telemetry

Motor control, queue manipulation, configuration writes, error clearing, and Wi-Fi control remain disabled.

“Observe” is more precise than “read-only”: heartbeat packets do change the controller’s daisy-link bookkeeping and activate periodic reporting, even though they do not intentionally alter motor operation.

No firmware modification or modernization of the old TCP library is required for this design.

3. Software architecture

A deliberately small implementation can have five components:

```
Serial transport
    | incoming byte chunks
    ▼
Incremental frame parser
    | validated frames
    ▼
Daisy-master session
    ├── discovery and heartbeat
    ├── topology tracking
    ├── SYNC request matching
    └── identity/profile selection
```

```
            |
            ▼
stable-002 telemetry decoder
            |
            ▼
Console summaries + raw capture log
```

Transport

Use a single owner for the serial device and a single serialized transmit path. Reads may return arbitrary byte chunks; neither a UART read nor an operating-system read corresponds to one protocol frame.

Proposed transport settings:

```
baud rate:       512000
data/parity/stop: 8N1, pending on-wire confirmation
flow control:    none
read behavior:   bounded/nonblocking
write behavior:  bounded, complete-frame writes
```

The firmware explicitly selects 512000 baud; 8N1 follows the default behavior expected from its `Serial.begin(...)` call and should be confirmed against the installed firmware.[1]

For a GPIO UART, use an appropriately configured PL011 rather than relying on the mini UART's clock-dependent baud rate. Actual device routing depends on the Pi model and configuration, so the program should accept a device path rather than assume `/dev/ttyAMA0` or `/dev/serial0`.[3]

4. Frame parser and transmission rules

The stable-002 header remains:

```
offset  size
0       1     magic = 0xAB
1       1     header checksum
2       1     payload checksum
3       1     target
4       1     queue
5       4     command ID, little-endian
9       1     opcode
10      2     payload length, little-endian
12      ...   payload
```

The payload checksum is the sum of payload bytes modulo 256. The header checksum is the sum of header bytes 2–11 modulo 256.[1]

Parser design

The host parser should:

1. Find a candidate `0xAB`.
2. Wait for the complete 12-byte header.
3. Validate the header checksum.
4. Check the declared length against a bounded host limit.
5. Wait for the complete payload.
6. Validate the payload checksum.
7. Emit the frame, preserving its original bytes.
8. On invalid data, advance cautiously and resume scanning.

Two additional protections matter:

- An incomplete-frame deadline prevents a plausible but corrupt header from blocking reception indefinitely.
- Unexpected opcode/length combinations are recorded rather than silently accepted as known telemetry.

For the initial observation profile, a conservative maximum total frame size of 2048 bytes matches the firmware's UART buffer capacity. Known telemetry should then be checked against much smaller profile-specific lengths. The 16-bit length field is not permission to send 64-KiB packets.[1]

Nonprotocol bytes should be retained as diagnostic data, not discarded without trace. Firmware initialization can print text on the same serial channel.

5. Session state machine

I recommend this state model:

```
CLOSED
   ↓ open serial device
CAPTURING
   ↓ explicitly enable discovery
DISCOVERING
   ↓ repeated consistent returned pings
TOPOLOGY_KNOWN
   ↓ send SYNC
IDENTIFYING
   ↓ compatible identity and validated layout
OBSERVING
```

Separate degraded states should cover:

```
LINK_STALE
TOPOLOGY_CHANGED
```

```
UNSUPPORTED_PROFILE
SERIAL_FAILURE
```

Discovery behavior

Send the canonical ping approximately every 250 ms:

```
AB 00 00 00 00 00 00 00 00 00 00 00
```

Through one slave, the expected returned frame is:

```
AB FF 00 FF 00 00 00 00 00 00 00 00
```

The target decrements as the ping traverses each slave. This is how the original master discovers the chain length.[1]

Require several consistent returned pings before accepting topology. A returned target of zero indicates a loop with no slave decrement, not a discovered controller.

Synchronization

After discovery, send one `SYNC` at a time. For target 1 and command ID 1:

```
AB 04 00 01 00 01 00 00 00 02 00 00
```

Register the pending request before transmission. Match the reply using command ID, expected opcode, and originating board—not ID alone.

Do not send host acknowledgments for unsolicited state or configuration packets: the inspected firmware does not require them. Maintain reception and heartbeat instead.[1]

6. A concrete stable-002 telemetry profile

I modeled the source structures using these explicit ABI assumptions:

- Little-endian integers.
- Four-byte `int`, `long`, and ordinary enums.
- One-byte Boolean values and explicitly packed enums.
- Four-byte IEEE floating-point values.
- Natural alignment inside nested structures that are not themselves packed.

That last point is easy to miss: packing an outer structure does not recursively remove padding inside its nested structures.

Predicted payload sizes

UART opcode	Meaning	Predicted payload length
0x00	Discovery ping	0
0x01	ACK	0
0x02	SYNC reply	315 bytes
0x03	Configuration report	228 bytes
0x04	State report	87 bytes

These are calculated expectations from the release-associated source, not measurements from your installed binary. They should become compatibility assertions only after capture confirms them.[1]

State payload layout

Under those assumptions:

Offset	Length	Block
0	15	Error state
15	12	Command state
27	22	Wi-Fi state
49	30	Motor state
79	8	Software signal

The motor-state block begins with 14 one-byte status/alarm fields, followed by speed, position, mark, and input-voltage values. Position and mark are signed 32-bit values.[1]

The configuration prefix provides a particularly robust initial decoder:

```
0-35      product
36-71     model
72-107    branch
108-109   version, uint16 little-endian
```

The expected identity is:

```
product: Wi-Fi Stepper
model:   wsx100
branch:  stable
version: 2
```

Decode and validate this prefix before attempting the motor-configuration body.[1]

A subtle SYNC limitation

The version-2 `SYNC` handler zero-initializes its response and copies several state blocks, but does not copy the software-signal state. The periodic `STATE` handler does copy it.

Therefore, the signal value in a SYNC snapshot should not be treated as authoritative. Use a subsequent periodic state report for that field.[1]

7. Compatibility and recovery policies

The first host should fail conservatively:

Event	Host response
Unexpected firmware identity	Preserve raw frames; disable profile-dependent interpretation
Unexpected payload length	Report mismatch; do not guess offsets
Missed heartbeat return	Mark link stale; preserve last-known data as stale
Changed slave count	Invalidate board-to-position mapping and re-identify
Serial reconnection	Restart discovery; do not replay commands
Corrupt frame	Record error and resynchronize
Request timeout	Record unknown outcome; no automatic state-changing retry

For multiple boards, use hardware identity from telemetry to associate a controller with a physical role. A target number is a ring position and can change when the chain changes.

The existing return-address relationship remains:

$$\text{originating board index} = N + \text{signed return target}$$

where N is the current discovered number of slaves.[1]

8. Revision D hardware: what is and is not established

Your revision D identification is now part of the design baseline. However, I have not established a revision-D-specific schematic-to-header mapping.

The release-era source snapshot does not contain the `hardware` directory found in the later repository snapshot. Consequently, the later schematic/netlist must not be assumed to describe your D boards exactly.

For now, the connection specification should remain signal-based:

```
Pi TX  → board RX
Pi RX  ← board TX
GND    ⇔ GND
```

The documented daisy-chain topology requires crossed transmit/receive signals and a shared ground.[4]

Before a physical trial, verify the D-board connector orientation, UART voltage levels, and whether a programmer or existing master is attached. Keep power rails separate unless an independently reviewed power arrangement requires otherwise. The present research does not justify connecting by guessed header position.

9. Safety and coexistence with REST

The stable-002 UART timeout path still does not stop motion: it clears communication bookkeeping. A lost heartbeat is a communication failure, not a motor interlock.[1]

The observation tool should therefore:

- Never issue an automatic stop merely because it exits.
- Never clear queues or errors during startup.
- Never turn off Wi-Fi.
- Never alter motor configuration.
- Never interpret stale telemetry as current.
- Clearly distinguish host shutdown from controller shutdown.

Keeping REST available is useful for comparing UART telemetry with the current workflow. For initial validation, avoid simultaneous state-changing clients; otherwise a position or command-state change cannot be attributed cleanly to the UART experiment.

10. First validation deliverable

The next implementation, when development becomes appropriate, should produce a capture —not a motor demonstration.

Its acceptance criteria should be:

1. Stable returned discovery pings from one D-board controller.
2. Reported identity `wsx100 / stable / 2`.
3. Observed configuration/state/SYNC lengths matching the selected ABI profile.
4. Plausible decoded status and position, cross-checked against REST while stationary.
5. Successful observation of heartbeat cessation and resumption without sending motor commands.
6. A complete timestamped record of transmitted frames, received bytes, parser failures, and session transitions.

I have also generated a small JSON design artifact containing the pinned source commit, ABI assumptions, predicted structure sizes, state-block offsets, and canonical packet vectors. It is a design/test fixture, not executable controller software or evidence of hardware validation.

⁂

1. https://github.com/goodrobotics/wifistepper

2. https://www.wifistepper.com/downloads/wsx100/check.html
3. https://www.raspberrypi.org/documentation/configuration/uart.md
4. https://www.wifistepper.com/daisy-chain.html
5. https://electricalflux.com/mcu-coding/uart-cable-pinout-color-code-reference
6. https://www.wifistepper.com/commands/configuration.html
7. https://github.com/goodrobotics/wifistepper/blob/master/documentation/src/interfaces/python.md
8. https://forum.dronebotworkshop.com/motors/wi-fi-stepper-wsx100-sw-behavior/
9. https://www.wifistepper.com/interfaces/low-level.html
10. https://github.com/smart-robotics
11. https://dronebotworkshop.com/wifi-stepper-motor-controller/
12. https://www.wifistepper.com/
13. https://www.wifistepper.com/interfaces/python.html
14. https://www.wifistepper.com/getting-started.html
15. https://github.com/Circuit-Digest/Wireless-Stepper-Motor-Controller
16. https://github.com/DFRobot/DFRobot_PCA
17. https://www.wifistepper.com/updates.html
18. https://forum.dronebotworkshop.com/motors/wifi-stepper-firmware/